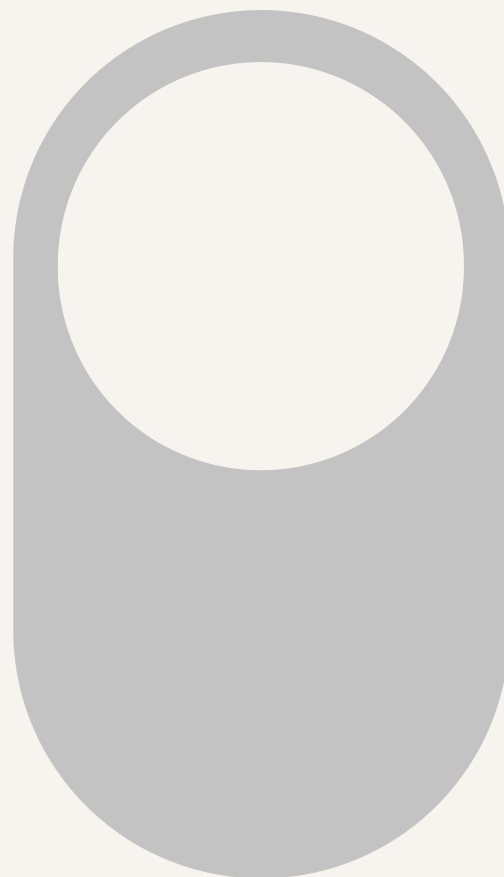




GUÍA PROFESIONAL · EDICIÓN 2026

Los 5 entregables de un desarrollo bien hecho

- 01 SRS — Especificación de requisitos
- 02 Tareas organizadas en Kanban
- 03 Desarrollo
- 04 MVP
- 05 Código fuente



Para quién

Quien encarga un desarrollo y quien lo entrega

Qué encontrarás

Qué es cada entregable, cómo se hace y cómo se conecta con el siguiente

Autor

Pablo Cabaleiro
Action Development

Emisión

Septiembre 2026

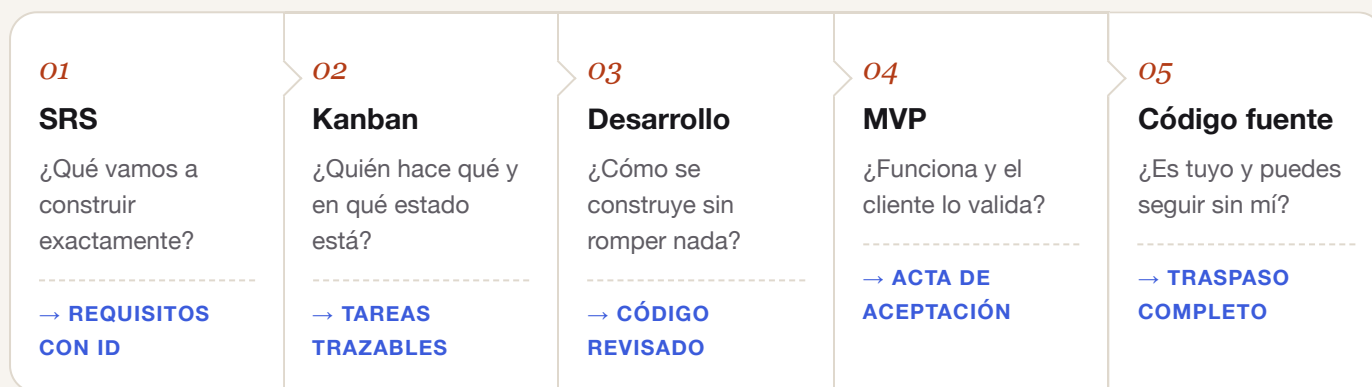


Un desarrollo no se entrega: se demuestra

"Te hago la app" no es un entregable. Un entregable es algo que el cliente puede abrir, revisar, firmar y quedarse.

La mayoría de proyectos que salen mal no fallan por el código. Fallan porque **nadie escribió qué había que construir**, nadie sabía en qué punto estaba cada cosa, y al final el cliente recibe algo que no esperaba, sin forma de comprobarlo y sin acceso real a lo que ha pagado.

En un desarrollo profesional se entregan **cinco cosas**, siempre en este orden. Y lo importante no es cada una por separado, sino que **están encadenadas**: la salida de una es la entrada de la siguiente. Si un eslabón falta, los demás pierden el sentido.



HILO CONDUCTOR

Para que se vea la conexión, seguiremos **un único requisito** a lo largo de toda la guía: **RF-07 · Recuperar contraseña por email**. Verás cómo nace en el SRS, se convierte en tarea, se programa, se valida en el MVP y acaba documentado en el código que se entrega.

LA REGLA DE ORO

Todo lo que se entrega tiene que poder **rastreadse hasta un requisito**, y todo requisito tiene que poder **rastreadse hasta algo entregado**. Si hay código que no responde a ningún requisito, sobra. Si hay un requisito sin código, falta. Esa es la conversación que evita el 90% de las discusiones con un cliente.

El contrato técnico del proyecto

El SRS es el documento que dice qué hace el software, para quién y con qué condiciones. Todo lo que viene después se construye encima de él.

No es un documento para rellenar expediente: es **la referencia contra la que se medirá la entrega**. Se redacta después de la reunión de kickoff, lo revisa el cliente y **se firma antes de escribir una sola línea de código**. La referencia de estructura es el estándar ISO/IEC/IEEE 29148 (sucesor del IEEE 830), pero se puede resumir en siete bloques.

1	Propósito y alcance Qué problema resuelve el producto, para quién y qué objetivo de negocio persigue. Un párrafo, sin tecnicismos.
2	Actores y roles Quién usa el sistema (usuario, administrador, invitado...) y qué puede hacer cada uno.
3	Requisitos funcionales (RF) Cada funcionalidad con ID único , descripción, prioridad y criterios de aceptación medibles.
4	Requisitos no funcionales (RNF) Rendimiento, seguridad, RGPD, accesibilidad, navegadores y dispositivos soportados. Con números: "carga en menos de 2 s", no "rápido".
5	Integraciones y dependencias APIs externas, pasarelas de pago, servicios de terceros y lo que el cliente debe aportar (accesos, contenidos, cuentas).
6	Fuera de alcance Lo que no se va a hacer, escrito explícitamente. Es la sección más importante del documento.
7	Glosario, supuestos y aprobación Términos del negocio, hipótesis de partida, versión del documento y firma de ambas partes.

FICHA DE REQUISITO

RF-07

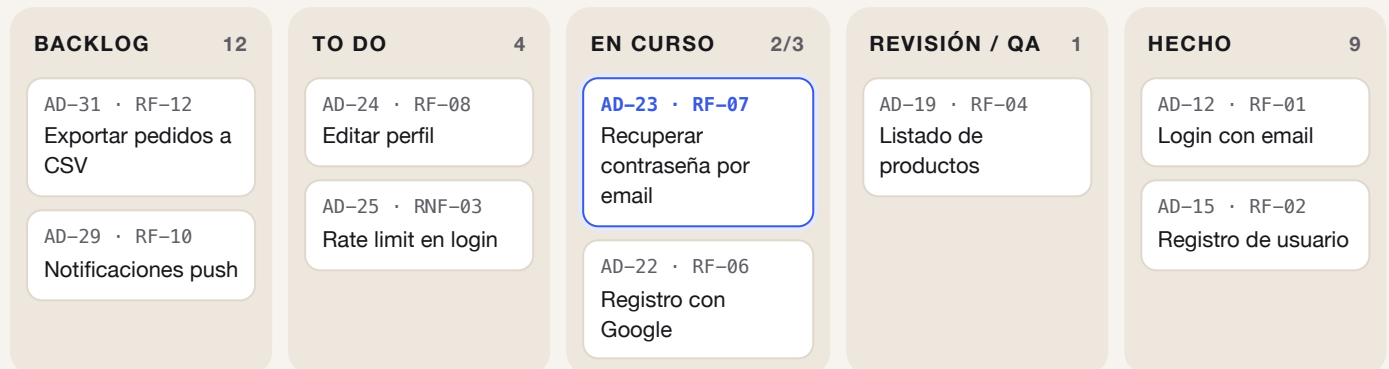
TÍTULO	Recuperar contraseña por email		
ACTOR	Usuario registrado		
PRIORIDAD	Must (imprescindible para el MVP)		
DESCRIPCIÓN	El usuario puede solicitar un enlace para restablecer su contraseña desde la pantalla de login.		
ACEPTACIÓN	DADO	un email registrado,	
	CUANDO	el usuario pulsa "He olvidado mi contraseña",	
	ENTONCES	recibe un enlace válido durante 30 minutos y de un solo uso.	

Por qué así: un criterio Dado / Cuando / Entonces se comprueba con un sí o un no. **Cómo se conecta:** cada RF con ID y prioridad es la materia prima del tablero Kanban; un SRS con IDs genera las tareas casi solo.

El proyecto, visible en una sola pantalla

El SRS dice qué hay que hacer. El Kanban dice qué se está haciendo ahora, qué falta y dónde está atascado.

Cada requisito del SRS se descompone en **tareas pequeñas** (idealmente de uno a tres días de trabajo) y cada tarea vive en una tarjeta que avanza por columnas. El tablero se comparte con el cliente en **modo lectura**: es el informe de avance que se actualiza solo.



ANATOMÍA

Qué lleva cada tarjeta

ID propio (AD-23), **requisito vinculado** (RF-07), criterios de aceptación copiados del SRS, estimación, responsable y etiquetas (front, back, bug, diseño).

LÍMITE WIP

No más de 3 en curso

El límite de trabajo en progreso obliga a terminar antes de empezar. Diez tareas "al 80%" no son nada entregable; tres terminadas, sí.

DEFINITION OF READY

Cuándo puede empezarse

Tiene requisito vinculado, criterios claros, diseño disponible si lo necesita y ninguna dependencia bloqueante.

DEFINITION OF DONE

Cuándo está terminada

Código revisado, tests en verde, desplegada en staging y criterios de aceptación comprobados. Si falta uno, no está en "Hecho".

HILO CONDUCTOR

RF-07 se convierte en la tarjeta AD-23, con los criterios Dado / Cuando / Entonces pegados en la descripción. Herramientas válidas: Linear, Jira, GitHub Projects o Trello. **La herramienta da igual; la disciplina no.**

Construir con un proceso, no a base de prisas

El desarrollo es la fase que el cliente no ve. Precisamente por eso tiene que dejar rastro de todo.

Un desarrollo profesional no se mide por lo rápido que se escribe código, sino por lo **predecible y revisable** que es. Cada tarea del tablero sigue siempre el mismo recorrido, y cualquier línea de código puede explicarse volviendo a su tarjeta y a su requisito.

```
# 1 · Una rama por tarea, con el ID de la tarjeta
git switch -c feature/AD-23-recuperar-password

# 2 · Commits convencionales que referencian la tarea
git commit -m "feat(auth): envío de enlace de recuperación (AD-23)"
git commit -m "test(auth): enlace caduca a los 30 min (AD-23)"

# 3 · Pull request → revisión + CI (lint, tests, build) → merge
gh pr create --title "AD-23 · RF-07 Recuperar contraseña"
```

ENTORNO	PARA QUÉ SIRVE	QUIÉN LO VE
Local	Donde se programa y se prueba cada tarea.	El equipo
Staging	Copia fiel de producción. Aquí se despliega todo lo que llega a "Hecho" y aquí se valida.	Equipo y cliente
Producción	Lo que usan los usuarios reales. Solo recibe versiones aprobadas.	Todo el mundo

CALIDAD

Revisión y tests
Nada llega a la rama principal sin pull request revisada y tests automáticos en verde.

SEGURIDAD

Secretos fuera del código
Claves y contraseñas en variables de entorno, nunca en el repositorio.

COMUNICACIÓN

Demo cada semana
El cliente ve avances reales en staging, no capturas ni promesas.

HILO CONDUCTOR

AD-23 tiene su rama, sus commits y su PR. Tras el merge se despliega en staging y la tarjeta pasa a "Hecho" solo cuando se han comprobado los criterios de RF-07 .

La primera versión que el cliente puede usar y validar

El MVP no es una versión "a medias". Es la versión más pequeña que ya resuelve el problema principal, completa y estable.

El alcance del MVP no se decide al final: **ya está en el SRS**, en la prioridad de cada requisito. Con el método **MoSCoW** (Must, Should, Could, Won't), todo lo marcado como *Must* forma el MVP. Lo demás va a iteraciones posteriores.

1 Congelar el alcance

Se cierra la lista de requisitos Must. Cualquier idea nueva a partir de aquí entra al backlog de la siguiente versión, no al MVP.

2 Entregar en staging

Todas las tarjetas del MVP en "Hecho", desplegadas en un entorno igual al de producción y con datos de prueba realistas.

3 Pruebas de aceptación (UAT)

El cliente recorre una checklist generada **directamente de los criterios del SRS**. Cada criterio: cumple o no cumple. Sin opiniones.

4 Corregir y volver a validar

Lo que no cumple se convierte en tarjeta de bug vinculada al mismo requisito y vuelve al tablero.

5 Acta de aceptación y salida a producción

Documento firmado que dice "el MVP cumple los requisitos X". A partir de ahí empieza el periodo de garantía pactado.

HILO CONDUCTOR

En la UAT, el cliente comprueba el caso **CA-07.1** : pide el enlace, lo recibe, cambia la contraseña y verifica que el mismo enlace ya no funciona una segunda vez.
Cumple → RF-07 aceptado.

EL ERROR MÁS CARO

Aceptar "solo una cosita más" dentro del MVP. Cada añadido sin pasar por el SRS retrasa la entrega, rompe la estimación y deja requisitos sin validar. Se apunta, se prioriza y va a la versión 1.1.

Lo que el cliente se queda cuando tú te vas

El último entregable es el que demuestra que el proyecto es del cliente: el código, la documentación y todos los accesos.

Entregar el código no es mandar un .zip. Es un **traspaso**: otro equipo tiene que poder clonar el repositorio, entender la arquitectura, arrancarlo en local y desplegarlo **sin tener que llamarte**. Si no puede, no has entregado el código: has entregado una dependencia.

REPOSITORIO

Propiedad transferida

El repositorio en la organización de GitHub/GitLab **del cliente**, con historial completo de commits y una versión etiquetada (`v1.0.0`).

DOCUMENTACIÓN

README y arquitectura

Cómo instalar, arrancar, testear y desplegar. Diagrama de arquitectura, modelo de datos y decisiones técnicas relevantes.

CONFIGURACIÓN

Variables de entorno

Un `.env.example` con todas las variables necesarias, explicadas, y sin ningún valor real dentro.

ACCESOS

Credenciales y cuentas

Dominio, hosting, base de datos, stores, APIs y correo transaccional, a nombre del cliente y entregados por un gestor de contraseñas, nunca por email.

Checklist de traspaso

- | | |
|---|--|
| ☐ Repositorio transferido y accesos del equipo revisados | ☐ Tag de versión y <code>CHANGELOG.md</code> actualizado |
| ☐ README probado por alguien que no ha tocado el proyecto | ☐ Pipeline de despliegue documentado y funcionando |
| ☐ Licencias de librerías de terceros revisadas | ☐ Propiedad intelectual cedida según contrato |
| ☐ Copia de seguridad de la base de datos y plan de restauración | ☐ Matriz de trazabilidad RF → tarea → PR → prueba |

HILO CONDUCTOR

`RF-07` cierra el círculo: aparece en el `CHANGELOG` de `v1.0.0`, el flujo está documentado en `docs/auth.md` y la matriz de trazabilidad lo enlaza con `AD-23`, su pull request y el caso de aceptación `CA-07.1`.

Un requisito, cinco entregables

Este es el recorrido completo de RF-07 . Si puedes rellenar esta tabla para cada requisito del proyecto, tu entrega es profesional.

ENTREGABLE	DÓNDE APARECE RF-07	QUÉ LO DEMUESTRA
01 · SRS	Ficha RF-07 con prioridad Must y criterios Dado / Cuando / Entonces	SRS firmado v1.0
02 · Kanban	Tarjeta AD-23 vinculada a RF-07	Tarjeta en "Hecho" con su DoD cumplido
03 · Desarrollo	Rama, commits y PR con AD-23	PR revisada y CI en verde
04 · MVP	Caso de aceptación CA-07.1 en staging	Acta de aceptación firmada
05 · Código	CHANGELOG v1.0.0 y docs/auth.md	Repositorio transferido al cliente

Los errores que rompen la cadena

SIN SRS

Empezar a programar tras la reunión
Sin requisitos escritos, cada desacuerdo se resuelve con la memoria de cada uno. Y siempre pierde alguien.

SIN TABLERO

Avances por WhatsApp
El cliente pregunta "¿cómo va?" porque no tiene dónde mirarlo. Un tablero compartido elimina esa pregunta.

SIN PROCESO

Subir directo a producción
Sin ramas, sin revisión y sin staging, cada cambio es una apuesta con el negocio del cliente.

SIN TRASPASO

Retener el código o los accesos
Un cliente que no puede irse no es un cliente fiel: es un cliente atrapado. Y lo acabará sabiendo.

Para terminar

Si solo lees esta página, quédate con esto.

1 Primero se escribe, luego se programa

El SRS con requisitos numerados y criterios medibles es la base de todo. Sin él, nada de lo demás se puede comprobar.

2 Todo avance tiene que ser visible

Cada requisito se convierte en tareas de un tablero Kanban compartido, y cada tarea en código revisado y desplegado en staging.

3 Se valida contra lo acordado

El MVP se acepta comprobando los criterios del SRS, uno a uno, y se cierra con un acta firmada.

4 El proyecto es del cliente

Código, documentación y accesos se transfieren completos. Un buen traspaso es la mejor carta de recomendación.

SRS, Kanban, desarrollo, MVP y código fuente. Cinco entregables, un solo hilo. Así trabajamos cada proyecto en Action Development.

Si tienes una idea y quieres que se desarrolle con este nivel de rigor —o te han entregado un proyecto y no sabes si le falta alguno de estos cinco puntos—, escíbeme y lo revisamos juntos.



Transforma tus ideas en ventas

CONTACTO

hi@actiondev.es

Respuesta en menos de 24h

ESTUDIO

Vigo, España

actiondev.es

SÍGUEME

Instagram — [@pabl](#)

Instagram — [@action.dev](#)